ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Midterm - Week 6 (Comprehensive Guide)

Instructor: Henry V. Dela Cruz

I. Overview and Objectives

This week, we are diving deep into how Python handles data in memory and how it presents that data to the user. We will cover the mechanics of output formatting and perform a deep dive into variables, naming rules, and dynamic typing.

Intended Learning Outcomes (ILOs):

- Utilize the `print()` function to output strings, numbers, and multiple variables.
- Apply Formatting: String Concatenation, the `.format()` method, and modern f-strings.
- Implement escape characters to control multi-line console output.
- Define the purpose of variables and execute variable declaration and assignment.
- Analyze dynamic typing in Python and apply strict naming conventions and constants.

II. Introduction to print() & Output Formatting

1. Printing Strings, Numbers, and Multiple Variables

The `print()` function is used to output data to the console. You can print multiple variables simultaneously by separating them with commas. Python automatically inserts a space between them.

```
print("System Online.")  # Printing a string
print(2026)              # Printing a number

user = "Admin"
status = "Active"
print("User:", user, "| Status:", status) # Printing multiple variables
```

2. Formatting Output: Concatenation, .format(), and f-strings

- **Concatenation (+):** Combines strings. You must manually cast numbers using `str()` to concatenate them.
- **.format():** Uses `{}` as placeholders in the string, injecting variables passed into the method.
- **f-strings:** The modern, most readable standard. Prefix the string with `f` and insert variables directly into the braces.

```
version = 3.1
# 1. Concatenation
print("App version is " + str(version))

# 2. .format()
print("App version is {}".format(version))

# 3. f-strings
print(f"App version is {version}")
```

3. Use of Escape Characters

Escape characters allow you to manipulate text layout (like adding tabs or newlines) within a single string.

Escape	Meaning	Example	Result
	New Line	`print("Top Bottom")`	Top (line break) Bottom
	Tab Space	`print("A B")`	A B

III. Deep Dive: Variables and Data Handling

1. Definition, Purpose, Declaration, and Assignment

A variable is a reserved memory location to store values. In Python, you do not need a command to declare a variable. A variable is created the moment you first assign a value to it using the = operator.

2. Dynamic Typing in Python

Python is dynamically typed. This means the type of the variable is checked during runtime, not before. A variable can even change its data type after it has been set.

```
x = 100       # x is an int
print(type(x))

x = "Active"  # x is now a str (dynamic typing in action)
print(type(x))
```

3. Naming Rules, Conventions, and Constants

- **Rules:** Must start with a letter or underscore. Cannot start with a number. Case-sensitive.
- **Convention:** Use *snake_case* for regular variables (e.g., `student_age`).
- **Constants & Best Practices:** Python does not have built-in constant types. As a best practice, if a variable's value should never change (like pi or a tax rate), write the variable name in **ALL_CAPS**.

```
# Variable Convention
first_name = "Juan"

# Constant Best Practice (Indicates this should not be changed)
MAX_CONNECTIONS = 100
PI_VALUE = 3.14159
```

Laboratory Coding Session: The Bakawan Profile Generator

Scenario: You are tasked with building a formatted console output for the Bakawan Data Analytics ecosystem. The program must declare variables using proper naming conventions, utilize a constant for a standard growth metric, and format a final output card using f-strings and escape characters.

Instructions:

1. Declare a constant named `BASE_GROWTH_RATE` and set it to `1.5`.
2. Ask the user for the `volunteer_name`, `site_zone`, and `saplings_planted`.
3. Demonstrate dynamic typing: Accept `saplings_planted` as a string, then cast it to an integer.
4. Calculate the `projected_yield` by multiplying the saplings by the `BASE_GROWTH_RATE`.
5. Use a single `print()` statement, heavy with and , to generate the formatted f-string output below.

Sample Program Run (Expected Output):

```
Enter Volunteer Name: Henry Dela Cruz
Enter Site Zone: Estuary Sector B
Enter Saplings Planted: 250

========================================
        BAKAWAN VOLUNTEER PROFILE
========================================
Name:           Henry Dela Cruz
Assigned Zone:  Estuary Sector B
Base Rate:      1.5
----------------------------------------
Planted:250
Proj. Yield:    375.0
========================================
```